ECE2002
DIGITAL LOGIC DESIGN

Module 5 – Lecture 1

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Hardware Description Language

- Computer-based language that describes the hardware of digital systems in a textual form.
- Resembles an ordinary computer programming language, such as C, but specifically oriented to describing hardware structures and the behavior of logic circuits.
- Used to represent logic diagrams, truth tables, Boolean expressions, and complex abstractions of the behavior of a digital system.
- An HDL description of an AND gate, for example, describes how the logic value of the gate's output is determined by the logic values of its inputs.
- The language content can be stored, retrieved, edited, and transmitted easily and processed by computer software in an efficient manner.
- Used in several major steps in the design flow of an integrated circuit: design entry, functional simulation or verification, logic synthesis, timing verification, and fault simulation.
- In the public domain, there are two standard HDLs that are supported by the IEEE: **VHDL and Verilog**.

Dr. Arindam Dutta

Hardware Description Language

- The language reference manual for the Verilog HDL presents a syntax that describes precisely the constructs that can be used in the language.

- A Verilog model is composed of text using keywords, of which there are about 100. Keywords are predefined lowercase identifiers that define the language constructs. Examples of keywords are **module**, **endmodule**, **input**, **output**, **wire**, **and**, **or**, **not**, etc.

- Any text after two forward slashes (//) and the end of the line is interpreted as a comment and will have no effect on a simulation using the model.

- Multiline comments begin with /* and terminate with */.

- Blank spaces are ignored, but they may not appear within the text of a keyword, a user-specified identifier, an operator, or the representation of a number.

- Verilog is case sensitive, which means that uppercase and lowercase letters are distinguishable (e.g., **not** is not the same as NOT).

- The term *module* refers to the text enclosed by the keyword pair **module** . . . **endmodule**. A module is the fundamental descriptive unit in the Verilog language. It is declared by the keyword **module** and must always be terminated by the keyword **endmodule**.

Dr. Arindam Dutta

Hardware Description Language

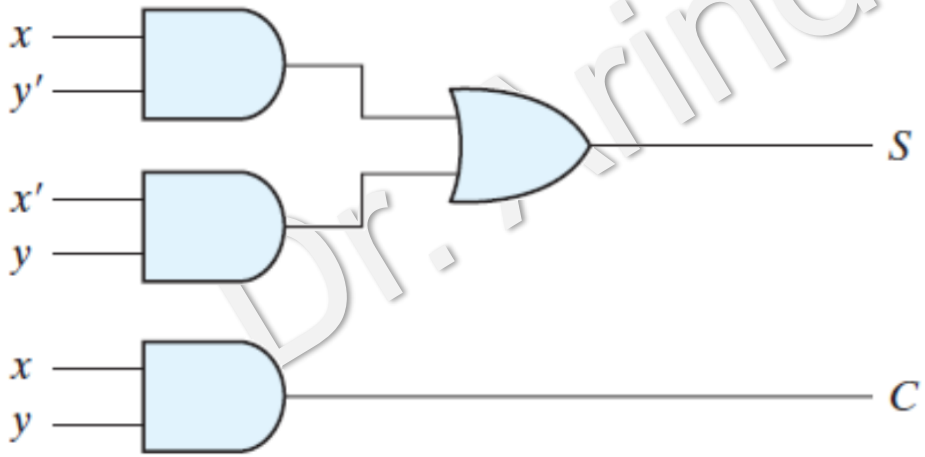
Combinational logic can be described by a schematic connection of gates, by a set of Boolean equations, or by a truth table. Each type of description can be developed in Verilog.

For example, half adder:.

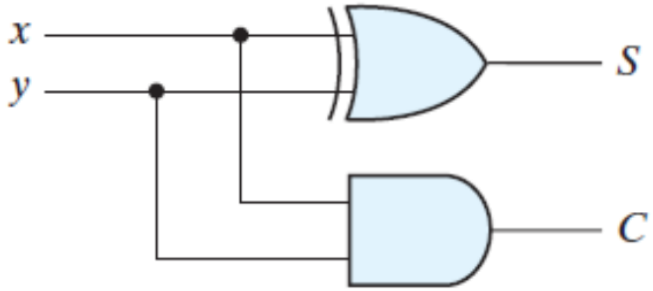
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Sum, $S = x'y + xy' = x \oplus y$

Carry, $C = x\,y$



(a) $S = xy' + x'y$
$C = xy$



(b) $S = x \oplus y$
$C = xy$

Dr. Arindam Dutta

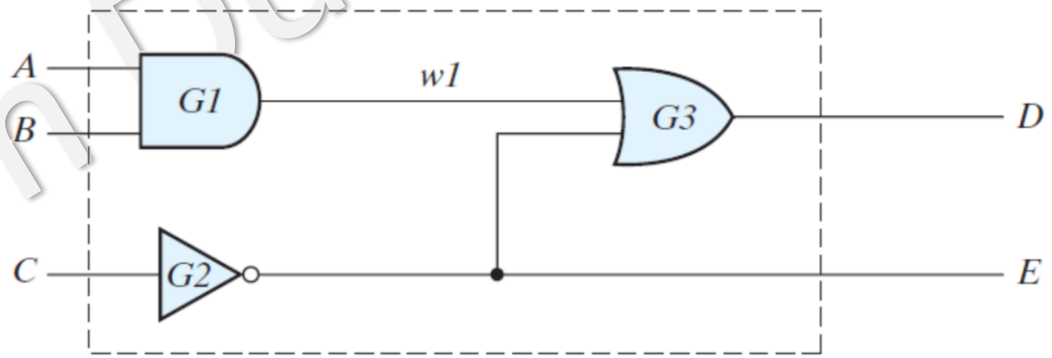
Hardware Description Language

Logic Gates

Example: Verilog gate-level description of combinational logic

```
// Verilog model of circuit. IEEE 1364–1995 Syntax

module Simple_Circuit (A, B, C, D, E);
output D, E;
input A, B, C;
wire w1;
and G1 (w1, A, B); // Optional gate instance name
not G2 (E, C);
or G3 (D, w1, E);
endmodule
```



Dr. Arindam Dutta

- The first line of text is a comment (optional) providing useful information to the reader.
- The second line begins with the keyword **module** and starts the declaration (description) of the module; the last line completes the declaration with the keyword **endmodule**.

Hardware Description Language

- The keyword **module** is followed by a name and a list of ports.
- The name (*Simple_Circuit* in this example) is an identifier. Identifiers are names given to modules, variables (e.g., a signal), and other elements of the language so that they can be referenced in the design.
- Identifiers are composed of alphanumeric characters and the underscore (_), and are case sensitive. Identifiers must start with an alphabetic character or an underscore, but they cannot start with a number.
- The *port list* of a module is the interface between the module and its environment. In this example, the ports are the inputs and outputs of the circuit.
- The port list is enclosed in parentheses, and commas are used to separate elements of the list.
- The statement is terminated with a semicolon (;).
- The keywords **input** and **output** specify which of the ports are inputs and which are outputs.
- Internal connections are declared as wires. The circuit in this example has one internal connection, at terminal *w1*, and is declared with the keyword **wire.**

Dr. Arindam Dutta

Hardware Description Language

- The structure of the circuit is specified by a list of (predefined) *primitive* gates, each identified by a descriptive keyword (**and, not, or**).

- The elements of the list are referred to as *instantiations* of a gate, each of which is referred to as a *gate instance*.

- Each *gate instantiation* consists of an optional name (such as *G1*, *G2*, etc.) followed by the gate output and inputs separated by commas and enclosed within parentheses.

- The output of a primitive gate is always listed first, followed by the inputs. For example, the OR gate of the schematic is represented by the **or** primitive, is named *G3*, and has output *D* and inputs *w1* and *E*.

- The module description ends with the keyword **endmodule.**

- Each statement must be terminated with a semicolon, but there is no semicolon after **endmodule.**

Dr. Arindam Dutta

Hardware Description Language

Boolean Expressions: Boolean equations describing combinational logic are specified in Verilog with a continuous assignment statement consisting of the keyword **assign** followed by a Boolean expression.

To distinguish arithmetic operators from logical operators, Verilog uses the symbols (&&), (||), (!), (~&), (~|), (^), and (^~/~^) for AND, OR, NOT, NAND, NOR, XOR, and XNOR respectively. Thus, to describe the simple circuit of Slide 4 with a Boolean expression, we use:

assign D = (A && B) | | (!C);

The example below describes a circuit that is specified with the following two Boolean expressions: ***E = A + BC + B'D*** and ***F = B'C + BC'D'***. The equations specify how the logic values *E* and *F* are determined by the values of *A, B, C,* and *D* .

Example: Combinational Logic Modeled with Boolean Equations

```
// Verilog model: Circuit with Boolean expressions

module Circuit_Boolean_CA (E, F, A, B, C, D);
output    E, F;
input     A, B, C, D;
assign E = A || (B && C) || ((!B) && D);
assign F = ((!B) && C) || (B && (!C) && (!D));
endmodule
```

Dr Arindam Dutta

Hardware Description Language

- The circuit has two outputs *E* and *F* and four inputs *A, B, C,* and *D*.
- The two **assign** statements describe the Boolean equations.
- The values of *E* and *F* during simulation are determined dynamically by the values of *A*, *B*, *C*, and *D*.
- An HDL description that provides the stimulus to a design is called a *test bench*.
- The simulator detects when the test bench changes a value of one or more of the inputs. When this happens, the simulator updates the values of *E* and *F*.
- The continuous assignment mechanism is so named because the relationship between the assigned value and the variables is permanent.
- The mechanism acts just like combinational logic, has a gate-level equivalent circuit, and is referred to as *implicit combinational logic*.

Dr. Arindam Dutta

Hardware Description Language

Truth Tables: The logic gates used in Verilog descriptions with keywords **and, or,** etc., are defined by the system and are referred to as *system primitives*. The user can create additional primitives by defining them in tabular form. These types of circuits are referred to as *user-defined primitives* (UDPs).

One way of specifying a digital circuit in tabular form is by means of a truth table. UDP descriptions do not use the keyword pair **module** . . . **endmodule**. Instead, they are declared with the keyword pair **primitive** . . . **endprimitive**.

They proceed according to the following general rules:

- It is declared with the keyword **primitive**, followed by a name and port list.
- There can be only one output, and it must be listed first in the port list and declared with keyword **output**.
- There can be any number of inputs. The order in which they are listed in the **input** declaration must conform to the order in which they are given values in the table that follows.
- The truth table is enclosed within the keywords **table** and **endtable**.
- The values of the inputs are listed in order, ending with a colon (:). The output is always the last entry in a row and is followed by a semicolon (;).
- The declaration of a UDP ends with the keyword **endprimitive**.

Dr. Arindam Dutta

Hardware Description Language

Example: User-Defined Primitive

```
// Verilog model: User-defined Primitive
primitive  UDP_02467 (D, A, B, C);
  output  D;
  input    A, B, C;
//Truth table for D 5 f (A, B, C) 5 Σ(0, 2, 4, 6, 7);
  table
//      A     B     C     :     D     // Column header comment
        0     0     0     :     1;
        0     0     1     :     0;
        0     1     0     :     1;
        0     1     1     :     0;
        1     0     0     :     1;
        1     0     1     :     0;
        1     1     0     :     1;
        1     1     1     :     1;
  endtable
endprimitive
```

Dr. Arindam Dutta

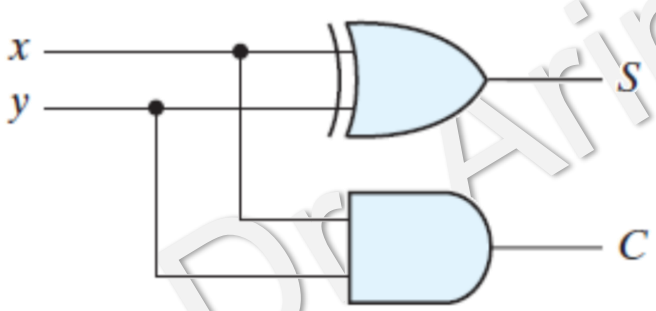
The variables listed on top of the table are part of a comment and are shown only for clarity. The system recognizes the variables by the order in which they are listed in the input declaration.

Hardware Description Language

Example: Gate-level HDL for half adder

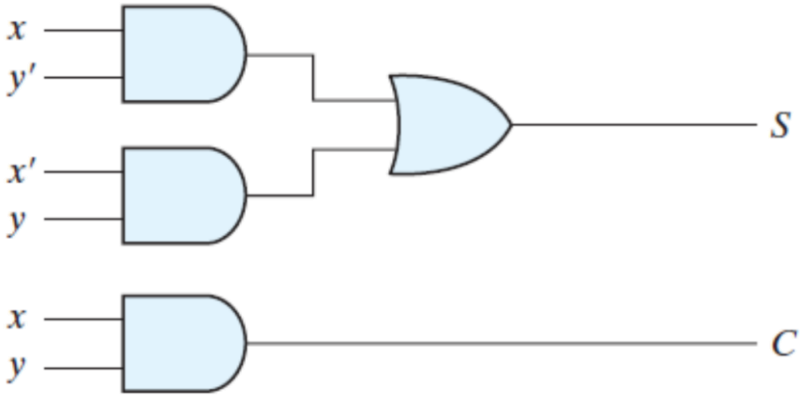
```
// Verilog model of half adder

module Half_Adder (x, y, S, C);
output S, C;
input x, y;
xor G1 (S, x, y);
and G2 (C, x, y);
endmodule
```




```
// Verilog model of half adder using basic gates

module Half_Adder (x, y, S, C);
output S, C;
input x, y;
wire w1, w2, w3, w4;
not G1 (w1, x);
not G2 (w2, y);
and G3 (w3, x, w2);
and G4 (w4, w1, y);
and G5 (C, x, y);
or G6 (S, w3, w4);
endmodule
```



Dr Arindam Dutta

Hardware Description Language

Example: HDL for half adder using Boolean expressions

```
// Verilog model of half adder

module Half_Adder (x, y, S, C);
output S, C;
input x, y;
assign S = x ^ y;
assign C = x && y;
endmodule
```

```
// Verilog model of half adder using basic gates

module Half_Adder (x, y, S, C);
output S, C;
input x, y;
assign S = (x && (!y)) || ((!x) && y);
assign C = x && y;
endmodule
```

Dr. Arindam Dutta

Hardware Description Language

Example: HDL for half adder using truth table

```
// Verilog model of half adder Sum

primitive HA_Sum (S, x, y);
output S;
input x, y;
table
// A  B : S
   0  0 :  0;
   0  1 :  1;
   1  0 :  1;
   1  1 :  0;
endtable
endprimitive
```

```
// Verilog model of half adder Carry

primitive HA_Carry (C, x, y);
output C;
input x, y;
table
// A  B : C
   0  0 :  0;
   0  1 :  0;
   1  0 :  0;
   1  1 :  1;
endtable
endprimitive
```

Dr. Arindam Dutta

ECE2002
DIGITAL LOGIC DESIGN

Module 5 – Lecture 2

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Hardware Description Language

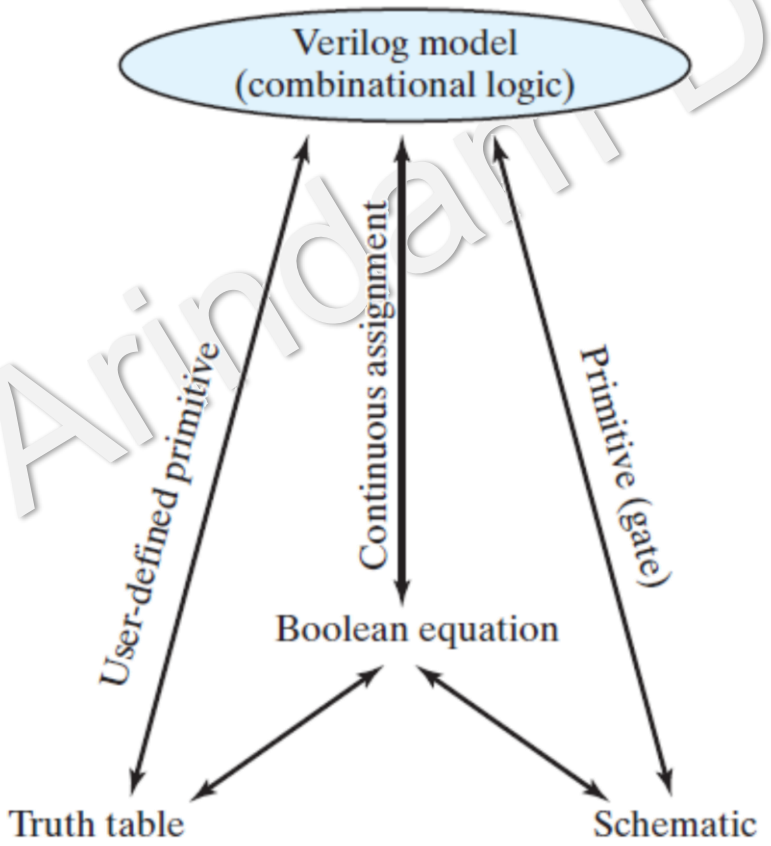
- Module is the basic building block for modeling hardware with the Verilog HDL.
- The logic of a module can be described in any one (or a combination) of the following modeling styles:
  - **Gate-level modeling** using instantiations of predefined and user-defined primitive gates.
  - **Dataflow modeling** using continuous assignment statements with the keyword **assign**.
  - **Behavioral modeling** using procedural assignment statements with the keyword **always**.
- Gate-level (structural) modeling describes a circuit by specifying its gates and how they are connected with each other.
- Dataflow modeling is used mostly for describing the Boolean equations of combinational logic.
- Behavioral modeling that is used to describe combinational and sequential circuits at a higher level of abstraction.

Dr. Arindam Dutta

Hardware Description Language

- Combinational logic can be designed with truth tables, Boolean equations, and schematics.

- Verilog has a construct corresponding to each of these "classical" approaches to design: user-defined primitives, continuous assignments, and primitives, as shown in the figure.



Dr. Arindam Dutta

Gate Level Modelling

- Here, a circuit is specified by its logic gates and their interconnections.
- Gate-level modeling provides a textual description of a schematic diagram.
- The Verilog HDL includes 12 basic gates as predefined primitives. They are all declared with the lowercase keywords **and**, **nand**, **or**, **nor**, **xor**, **xnor**, **not**, and **buf**.
- When a primitive gate is listed in a module, we say that it is *instantiated* in the module.
- In general, component instantiations are statements that reference lower level components in the design, essentially creating unique copies (or *instances*) of those components in the higher level module.
- A module that uses a gate in its description is said to *instantiate* the gate.
- Think of instantiation as the HDL counterpart of placing and connecting parts on a circuit board.

Dr. Arindam Dutta

Gate Level Modelling

Full Adder

```
module Full_Adder (x, y, z, S, C);

input x, y, z;
output S, C;

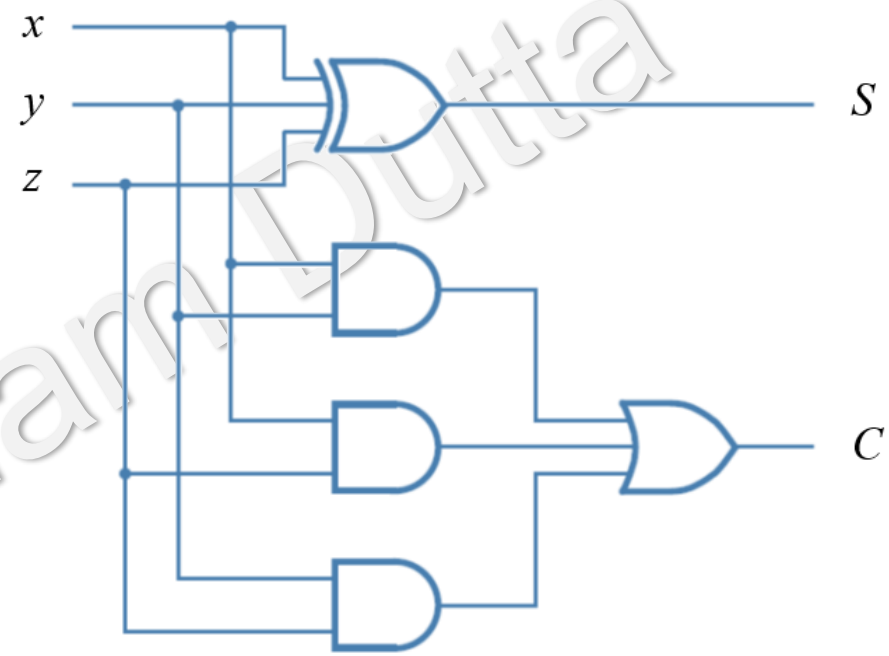
wire w1, w2, w3;

xor G1 (S, x, y, z);

and G2 (w1, x, y);
and G3 (w2, x, z);
and G4 (w3, y, z);

or G5 (C, w1, w2, w3);

endmodule
```



Dr. Arindam Dutta

Gate Level Modelling

Full Adder Using Two Half Adders and OR Gate

```
module Full_Adder (x, y, z, S, C);

input x, y, z;
output S, C;

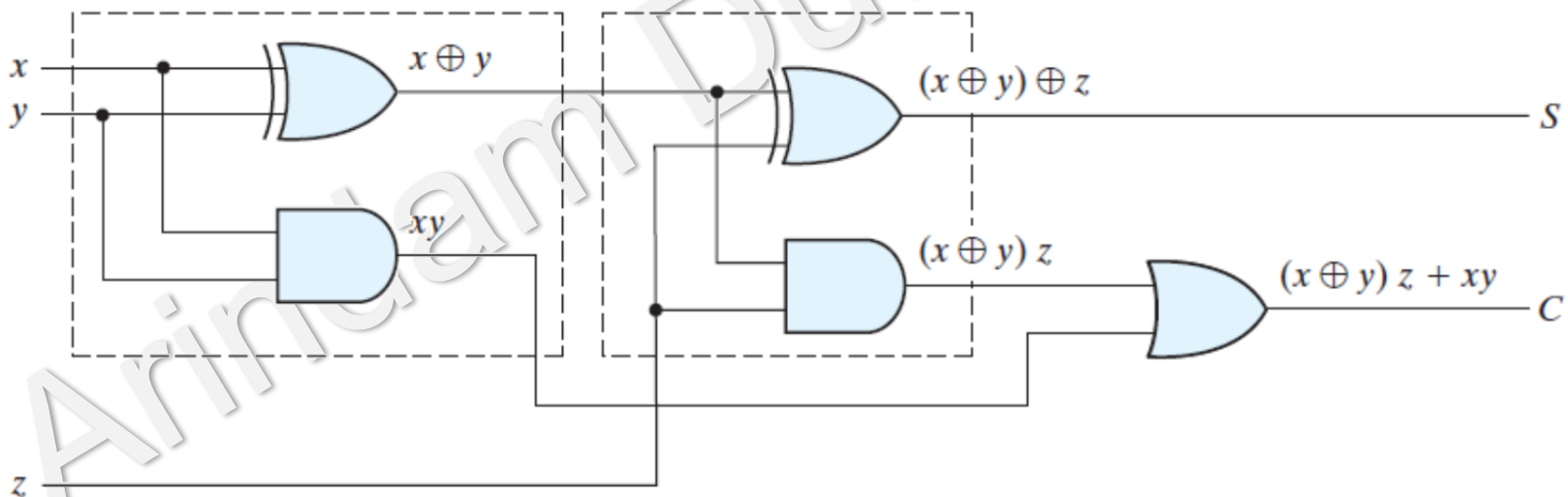
wire w1, w2, w3;

// First half adder
xor G1 (w1, x, y);
and G2 (w2, x, y);

// Second half adder
xor G3 (S, w1, z);
and G4 (w3, w1, z);

// OR gate for carry
or G5 (C, w2, w3);

endmodule
```

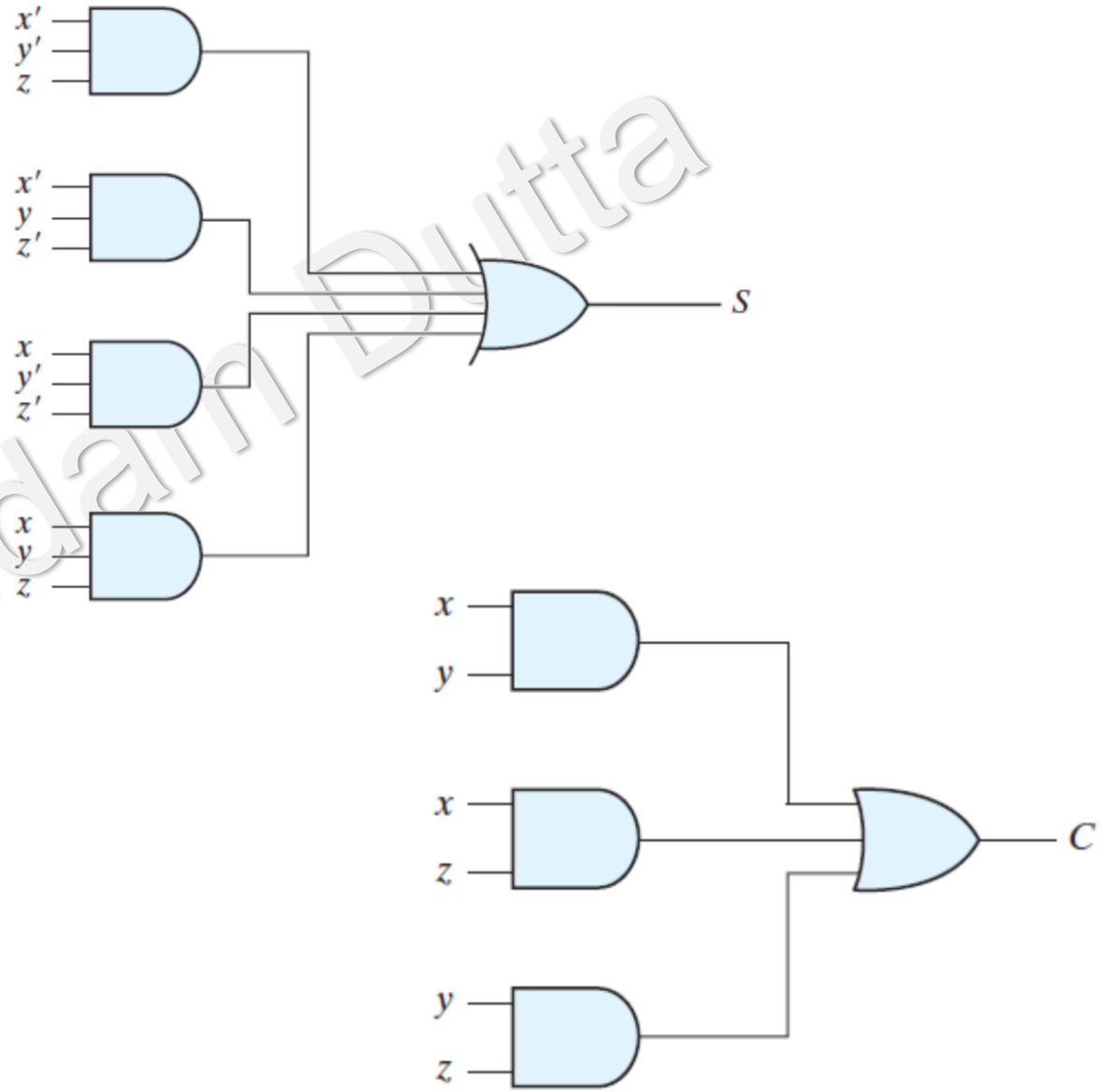


Dr. Arindam Dutta

Gate Level Modelling

Full Adder Using only Basic Gates

```
module Full_Adder (x, y, z, S, C);
input x, y, z;
output S, C;
wire w1, w2, w3, w4, w5, w6, w7, w8, w9, w10;
not G1 (w1, x);
not G2 (w2, y);
not G3 (w3, z);
and G4 (w4, x, w2, w3);
and G5 (w5, w1, y, w3);
and G6 (w6, w1, w2, z);
and G7 (w7, x, y, z);
or G8 (S, w4, w5, w6, w7);
and G9  (w8, x, y);
and G10 (w9, x, z);
and G11 (w10, y, z);
or G12 (C, w8, w9, w10);
endmodule
```



Dr. Arindam Dutta

Gate Level Modelling

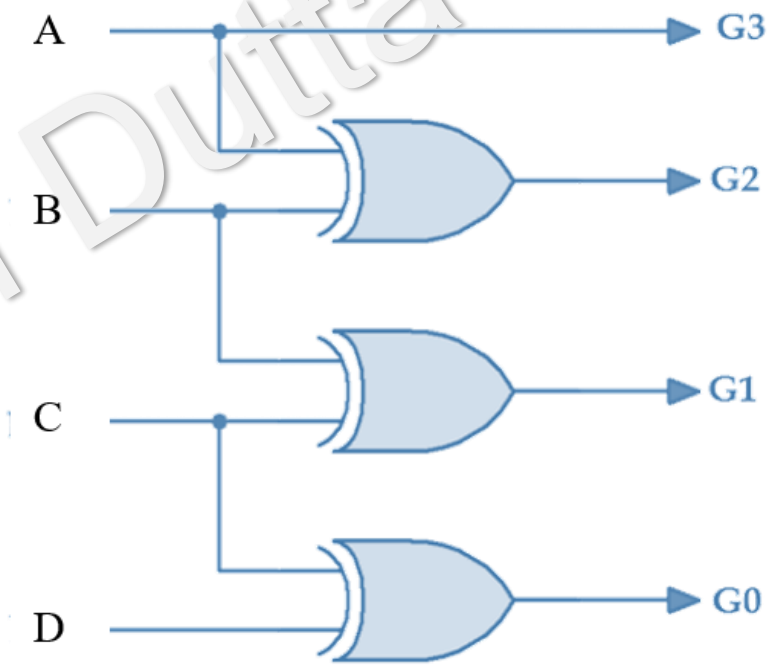
Binary to Gray Code Converter

```
module Binary_to_GrayCode (A, B, C, D, G3, G2, G1, G0);

input A, B, C, D;
output G3, G2, G1, G0;

buf B1 (G3, A);
xor X1 (G2, A, B);
xor X2 (G1, B, C);
xor X3 (G0, C, D);

endmodule
```



Dr. Arindam Dutta

Gate Level Modelling

Two-to-Four-Line Decoder with Enable

```
module Decoder_2to4 (I1, I0, E, Y3, Y2, Y1, Y0);

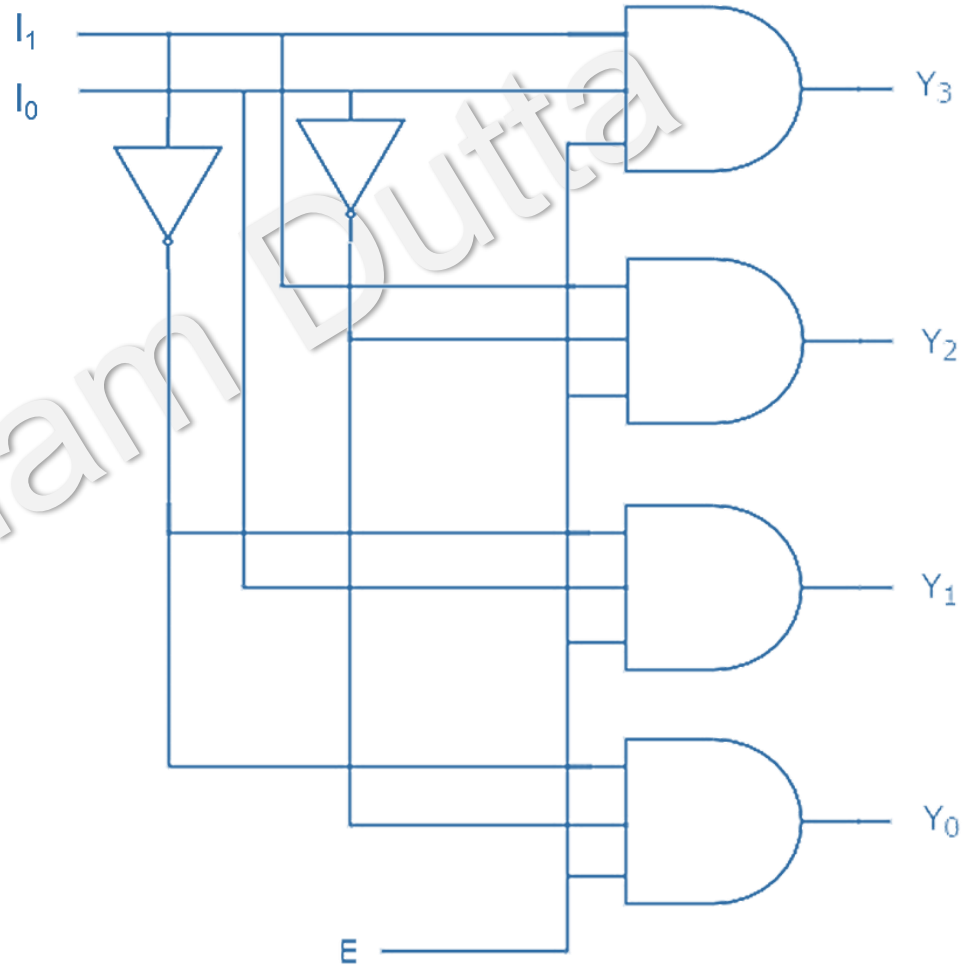
input I1, I0, E;
output Y3, Y2, Y1, Y0;

wire w1, w2;

not G1 (w1, I1);
not G2 (w2, I0);

and G3 (Y3, E, I1, I0);
and G4 (Y2, E, I1, w2);
and G5 (Y1, E, w1, I0);
and G6 (Y0, E, w1, w2);

endmodule
```



Dr. Arindam Dutta

Gate Level Modelling

Two-to-One Multiplexer

```
module MUX_2to1 (I0, I1, S, Y);

input I0, I1, S;
output Y;

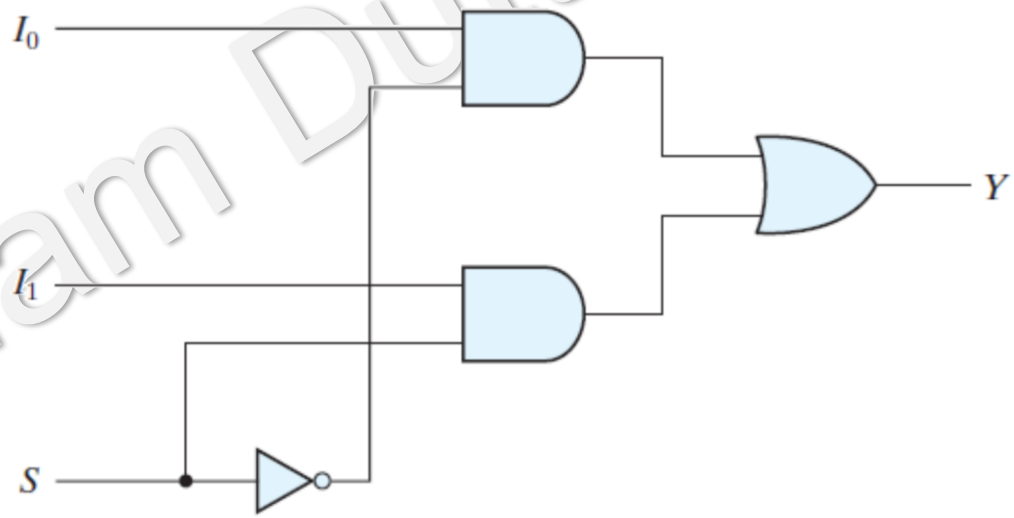
wire w1, w2, w3;

not G1 (w1, S);

and G2 (w2, I0, w1);
and G3 (w3, I1, S);

or G4 (Y, w2, w3);

endmodule
```



Dr. Arindam Dutta

Dataflow Modelling

- Dataflow modeling of combinational logic uses a number of operators that act on binary operands to produce a binary result.
- Verilog HDL provides about 30 different operators.
- It is necessary to distinguish between arithmetic and logic operations, so different symbols are used for each.

Symbol	Operation	Symbol	Operation
+	binary addition		
−	binary subtraction		
&	bitwise AND	&&	logical AND
|	bitwise OR	||	logical OR
^	bitwise XOR		
~	bitwise NOT	!	logical NOT
= =	equality		
>	greater than		
<	less than		
{}	concatenation		
?:	conditional		

Dr.Arindam Dutta

Dataflow Modelling

- The plus symbol (+) indicates the arithmetic operation of addition; the bitwise logic OR operation uses the symbol |.
- There are special symbols for bitwise logical AND, NOT, and XOR.
- The equality symbol uses two equals signs (without spaces between them) to distinguish it from the equals sign used with the **assign** statement.
- The bitwise operators operate bit by bit on a pair of vector operands to produce a vector result.
- The concatenation operator provides a mechanism for appending multiple operands.
- A bitwise operator (e.g., ~) and its corresponding logical operator (e.g., !) may produce different results, depending on their operand. For example, ~(1010) is (0101), and !(1010) is 0. A binary value is considered to be logically true if it is not 0.
- Usually, we use bitwise operators to describe arithmetic operations and logical operators to describe logical operations.
- Dataflow modeling uses continuous assignments and the keyword **assign.** A continuous assignment is a statement that assigns a value to a net.
- Dataflow HDL models describe combinational circuits by their *function* rather than by their gate structure.

Dr. Arindam Dutta

Dataflow Modelling

Full Adder

```
// Dataflow description for full adder
module Full_Adder (x, y, z, S, C);
input x, y, z;
output S, C;
assign S = x ^ y ^ z;
assign C = (x & y) | (x & z) | (y & z);
endmodule
```

Full Adder with Basic Gates

```
// Dataflow description of full adder with basic gates
module Full_Adder (x, y, z, S, C);
input x, y, z;
output S, C;
assign S = (x & ~y & ~z) | (~x & y & ~z) | (~x & ~y & z) | (x & y & z);
assign C = (x & y) | (x & z) | (y & z);
endmodule
```

Full Adder Using Two Half Adders and OR Gate

```
// Dataflow description for full adder using two half adders
module Full_Adder (x, y, z, S, C);
input x, y, z;
output S, C;
assign S = (x ^ y) ^ z;
assign C = ((x ^ y) & z) | (x & y);
endmodule
```

Dr. Arindam Dutta

Dataflow Modelling

Two-to-Four-Line Decoder with Enable

```
// Dataflow description of two-to-four-line decoder
module Decoder_2to4 (I1, I0, E, Y3, Y2, Y1, Y0);
input I1, I0, E;
output Y3, Y2, Y1, Y0;
assign Y3 = E & I1 & I0;
assign Y2 = E & I1 & ~I0;
assign Y1 = E & ~I1 & I0;
assign Y0 = E & ~I1 & ~I0;
endmodule
```

Two-to-One-Multiplexer

```
// Dataflow description of two-to-one-line multiplexer
module MUX_2to1 (I0, I1, S, Y);
input I0, I1, S;
output Y;
assign Y = (~S & I0) | (S & I1);
endmodule
```

Binary to Gray Code Converter

```
// Dataflow description of binary to gray code converter
module Binary_to_Gray (A, B, C, D, G3, G2, G1, G0);
input A, B, C, D;
output G3, G2, G1, G0;
assign G3 = A;
assign G2 = A ^ B;
assign G1 = B ^ C;
assign G0 = C ^ D;
endmodule
```

Dr. Arindam Dutta

Behavioural Modelling

- Behavioral modelling represents digital circuits at a functional and algorithmic level.
- It is used mostly to describe sequential circuits, but can also be used to describe combinational circuits.
- Behavioral descriptions use the keyword **always**, followed by an optional event control expression and a list of procedural assignment statements.
- The event control expression specifies when the statements will execute.
- The target output of a procedural assignment statement must be of the **reg** data type.
- Contrary to the **wire** data type, whereby the target output of an assignment may be continuously updated, a **reg** data type retains its value until a new value is assigned.

Dr. Arindam Dutta

Behavioural Modelling

Two-to-One-Multiplexer

```
// Behavioral description of 2-to-1 multiplexer

module mux_2to1 (Y, I0, I1, S);

    output Y;
    input I0, I1, S;
    reg Y;

    always @(I0 or I1 or S)
       if (S == 0)
          Y = I0;
       else
          Y = I1;

endmodule
```

S	Y
0	I_0
1	I_1

Four-to-One-Multiplexer

```
// Behavioral description of 4-to-1 multiplexer

module mux_4to1 (Y, I0, I1, I2, I3, S);

    output Y;
    input I0, I1, I2, I3;
    input [1:0] S;
    reg Y;

    always @(I0 or I1 or I2 or I3 or S)
       case (S)
          2'b00: Y = I0;
          2'b01: Y = I1;
          2'b10: Y = I2;
          2'b11: Y = I3;
       endcase

endmodule
```

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Dr. Arindam Dutta

Behavioural Modelling

- **For the 2:1 MUX**, since variable Y is a target output, it must be declared as **reg** data (in addition to the **output** declaration).
- The procedural assignment statements inside the **always** block are executed every time there is a change in any of the variables listed after the @ symbol.
- In this case, these variables are the input variables I0 , I1 , and S.
- The statements execute if I0, I1, or S changes value.
- There is no semicolon (;) at the end of the **always** statement.
- The keyword **or**, instead of the bitwise logical OR operator "|", is used between variables.
- The conditional statement **if–else** provides a decision based upon the value of the select input. The **if** statement can be written without the equality symbol if the value of the variable (within the braces) is 1:

if (S) Y = I1;

The statement implies that select is checked for logic 1.

Dr. Arindam Dutta

Behavioural Modelling

- **For the 4:1 MUX**, the S input is defined as a two-bit vector, and output Y is declared to have type **reg**.
- The **always** statement has a sequential block enclosed between the keywords **case** and **endcase**.
- The block is executed whenever any of the inputs listed after the **@** symbol changes in value.
- The **case** statement is a multiway conditional branch construct.
- Whenever I0, I1, I2, I3 or S change, the case expression (S) is evaluated and its value compared, from top to bottom, with the values in the list of statements that follow, the so-called **case** items.
- The statement associated with the first **case** item that matches the **case** expression is executed.
- In the absence of a match, no statement is executed. Since select is a two-bit number, it can be equal to 00, 01, 10, or 11.
- The **case** items have an implied priority because the list is evaluated from top to bottom.
- Binary numbers in Verilog are specified and interpreted with the letter **b** preceded by a prime.
- The size of the number is written first and then its value. Thus, 2'b01 specifies a two-bit binary number whose value is 01.

Dr. Arindam Dutta

Behavioural Modelling

- Numbers are stored as a bit pattern in memory, but they can be referenced in decimal, octal, or hexadecimal formats with the letters ***'d***, ***'o***, and ***'h***, respectively.
- For example, 4'hA = 4'd10 = 4'b1010 and have the same internal representation in a simulator.
- If the base of the number is not specified, its interpretation defaults to decimal.
- If the size of the number is not specified, the system assumes that the size of the number is at least 32 bits.
- If a host simulator has a larger word length (say, 64 bits) the language will use that value to store unsized numbers.

Dr. Arindam Dutta

Behavioural Modelling

Full Adder

```
// Behavioral description of full adder
module Full_Adder (S, C, x, y, z);
    output S, C;
    input x, y, z;
    reg S, C;
    always @(x or y or z)
        case ({x, y, z})
            3'b000: S = 0, C = 0;
            3'b001: S = 1, C = 0;
            3'b010: S = 1, C = 0;
            3'b011: S = 0, C = 1;
            3'b100: S = 1, C = 0;
            3'b101: S = 0, C = 1;
            3'b110: S = 0, C = 1;
            3'b111: S = 1, C = 1;
        endcase
endmodule
```

Two-to-Four-Line Decoder with Enable

```
// Behavioral description of 2to4 decoder
module Decoder_2to4 (I1, I0, E, Y3, Y2, Y1, Y0);
input I1, I0, E;
output Y3, Y2, Y1, Y0;
reg Y3, Y2, Y1, Y0;
always @(I1 or I0 or E)
    case ({E,I1,I0})
        3'b000: Y3 = 0, Y2 = 0, Y1 = 0, Y0 = 0;
        3'b001: Y3 = 0, Y2 = 0, Y1 = 0, Y0 = 0;
        3'b010: Y3 = 0, Y2 = 0, Y1 = 0, Y0 = 0;
        3'b011: Y3 = 0, Y2 = 0, Y1 = 0, Y0 = 0;
        3'b100: Y3 = 0, Y2 = 0, Y1 = 0, Y0 = 1;
        3'b101: Y3 = 0, Y2 = 0, Y1 = 1, Y0 = 0;
        3'b110: Y3 = 0, Y2 = 1, Y1 = 0, Y0 = 0;
        3'b111: Y3 = 1, Y2 = 0, Y1 = 0, Y0 = 0;
    endcase
endmodule
```

Dr. Arindam Dutta

Behavioural Modelling

Binary to Gray Code Converter

```
module Binary_to_Gray (A, B, C, D, G3, G2, G1, G0);

input A, B, C, D;
output G3, G2, G1, G0;
reg G3, G2, G1, G0;

always @(A or B or C or D)
   case ({A,B,C,D})
      4'b0000: G3 = 0, G2 = 0, G1 = 0, G0 = 0;
      4'b0001: G3 = 0, G2 = 0, G1 = 0, G0 = 1;
      4'b0010: G3 = 0, G2 = 0, G1 = 1, G0 = 1;
      4'b0011: G3 = 0, G2 = 0, G1 = 1, G0 = 0;
      4'b0100: G3 = 0, G2 = 1, G1 = 1, G0 = 0;
      4'b0101: G3 = 0, G2 = 1, G1 = 1, G0 = 1;
      4'b0110: G3 = 0, G2 = 1, G1 = 0, G0 = 1;
      4'b0111: G3 = 0, G2 = 1, G1 = 0, G0 = 0;
      4'b1000: G3 = 1, G2 = 1, G1 = 0, G0 = 0;
      4'b1001: G3 = 1, G2 = 1, G1 = 0, G0 = 1;
      4'b1010: G3 = 1, G2 = 1, G1 = 1, G0 = 1;
      4'b1011: G3 = 1, G2 = 1, G1 = 1, G0 = 0;
      4'b1100: G3 = 1, G2 = 0, G1 = 1, G0 = 0;
      4'b1001: G3 = 1, G2 = 1, G1 = 0, G0 = 1;
      4'b1010: G3 = 1, G2 = 1, G1 = 1, G0 = 1;
      4'b1011: G3 = 1, G2 = 1, G1 = 1, G0 = 0;
      4'b1100: G3 = 1, G2 = 0, G1 = 1, G0 = 0;
      4'b1101: G3 = 1, G2 = 0, G1 = 1, G0 = 1;
      4'b1110: G3 = 1, G2 = 0, G1 = 0, G0 = 1;
      4'b1111: G3 = 1, G2 = 0, G1 = 0, G0 = 0;
   endcase

endmodule
```

Dr Arindam Dutta